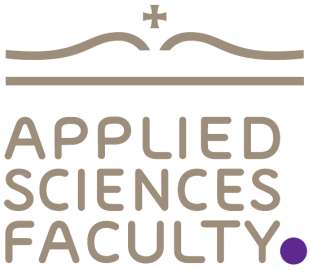
C++ — лекція 13. Класи — 3. Допоміжні засоби





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Перевантаження операторів

C++ надає можливість перевантажувати оператори **для користувацьких типів**, включаючи класи, структури, об'єднання, перерахування. Завдяки цьому можна реалізовувати для них природну нотацію:

```
class Vector2 {
public:
    double x,y;
        ..................
};
   ..............
Vector2 operator+(const Vector2& a, const Vector2& b){
    Vector2 res;
    res.x = a.x + b.x;
    res.y = a.y + b.y;
    return res;
}
   ..............
Vector2 a(1,3), b (3,4), c;

c = a + b;
```

Перевантажувати можна більшість операторів:

```
+     -     *     /     %     ^     &
|     ~     !     =     <     >     +=
-=    *=    /=    %=    ^=    &=    |=
<<    >>    >>==  <<==  ==    !=    <=
>=    &&    ||    ++    --    ->*   ,
->    []    ()
new         new[]     delete     delete[]
""<id> (C++11)  <=> (C++20)   co_await (C++20)
```

- **Увага!** Перевантажені оператори "&&", "||" і "," втрачають здатність скороченого/впорядкованого обчислення (до C++20 – подробиці див. cpp1_lecture02_statements.pdf).
- Заборонено (поки?) перевантажувати наступні оператори:

  ```
  ::          .          .*     ?:
  ```

  та іменовані оператори:

  ```
  sizeof    typeid    alingof (C++11) <etc...>
  ```

- **Не можна створювати свої оператори**.
- Дозвіл чи того чи того привів би до великої плутанини, не надаючи особливих переваг. Хіба що тернарний можна було б безпечно перевантажувати, але поки стандартизаційний комітет не бачить в тому потреби.

Використовуючи перевантаження, не порушуйте природної семантики відповідних операцій!

"***Не знаєте, як зробити, робіть так, як є для int***", Скот Меєрс

- Зокрема, для вбудованих типів, якщо реалізовано оператор +, то існує і оператор += та пара ++x, x++, однак для користувацьких така відповідність сама не досягається – про неї має подбати програміст.
- Так само, для вбудованих типів виконується a+b == b+a, тобто додавання комутативне. Цієї ж поведінки слід очікувати і від перевизначених операторів, там де це має сенс[1].
- Та й результат x + y - y повинен бути еквівалентним x.
- Поміж арифметичних операторів створюйте лише ті, що мають очевидний сенс для ваших типів.
  - ◇ Приклад: для вказівників існує операція додавання цілого числа, існує різниця двох вказівників, однак немає їх додавання.)
- Якщо оператор не змінює своїх аргументів для вбудованих типів, він не повинен їх змінювати і для вашого класу. (Приклад: x + y не може змінювати x чи y!)

[1] Скажімо, хоч і не зовсім коректним із сучасної точки зору, але історично поширеним та загально-вживаним оператором конкатенації стрічок є оператор "+", а очікувати від конкатенації комутативності аж ніяк не доводиться. Множення багатьох математичних об'єктів, скажімо — матриць, не є комутативним. Python має для таких випадків спеціальний оператор – @.

Використовуючи перевантаження, не порушуйте природної семантики відповідних операцій!

"***Не знаєте, як зробити, робіть так, як є для int***", Скот Меєрс

- Оператори виду z = x + y повинні повертати копію, інакше будуть неусувні проблеми із контролем за тим хто ж володіє об'єктом. І навпаки, x += y повинен змінювати x, але не y.
- Для уникнення дублювання коду, варто в класі реалізовувати оператори із присвоєнням, а за його межами — відповідні бінарні, використовуючи методи класу із присвоєнням. За такого підходу можна також добитися симетрії у використанні бінарних операторів[2]. Ще одна перевага такого підходу — зовнішні по відношенню до класу оператори не повинні бути його друзями.
- Кінцевий стан об'єкту, після ++x чи x++, повинен бути одним і тим же. Те ж стосується ++x і x += 1.
- Те ж стосується *p та p[0] (зрозуміло, якщо ви один з них, чи обидва, перевантажили), а p[i] і *(p+i), крім того, ще не мають права змінювати p.

[2]Наприклад, згаданої комутативності, про компілятор нічого не знає, особливо для додавання змінних різних типів. Приклади нижче.

Використовуючи перевантаження, не порушуйте природної семантики відповідних операцій!

"***Не знаєте, як зробити, робіть так, як є для int***", Скот Меєрс

- Якщо визначено оператор ==, то коли він повертає true для x==y, x та y повинні бути еквівалентні, тотожні за поведінкою, (настільки, наскільки це можливо і практично, а всі винятки слід акуратно документувати).
- Між == і != повинно бути природне співвідношення, результат x == y повинен співпадати з !(x != y).
- Таке ж співвідношення повинно бути між операторами порівняння, x < y має бути еквівалентним до !(x >= y).
- Оператори співвідношення, <, <=, >, >=, повинні підтримувати транзитивність, там де в цьому є сенс: якщо a<b та b<c, то a<c.
- **Взагалі, користуйтеся здоровим глуздом**.

Детальніше про це можна почитати в офіційному FAQ C++: `https://isocpp.org/wiki/faq/operator-overloading#op-ov-rules`, "What are some guidelines / "rules of thumb" for overloading operators?".

Вимоги до перевантажених операторів

Жорстких вимог небагато. Решта — на відповідальності програміста.

- Перевизначений оператор має бути методом користувацького класу, або містити хоча б один аргумент типу користувацького класу.
- `operator=`,`operator()`, `operator[]` і `operator->` мають бути нестатичними функціями-методами. Це гарантує, що їх перший аргумент буде lvalue.
  - ◇ C++23 дозволив створювати статичні `operator()` та `operator[]`.

Однак, дотримуйтеся природної семантики!

Перевантаження бінарних операторів

- Бінарний оператор може бути або нестатичним методом із одним аргументом, або незалежною функцією із двома аргументами.
- Тобто, використання довільного бінарного оператора `@`: `a @ b`, може інтерпретуватися як `a.operator@(b)`, або як `operator@(a, b)`:

```
class Vector2{
        ..........
    Vector2 operator+(const Vector2& b); // [a]
    Vector2 operator+(const double &b);  // [b]
};
Vector2 operator+(const double &a, const Vector2& b); // [c]
```

```
        ..............
    Vector x, y, z;
        ..............
    z = x + y;   // [a], z = x.operator+(y);
    z = x + 5.0; // [b], z = x.operator+(0.5);
    z = 0.5 + y; // [c], z = operator+(0.5, y)
```

А ось створити додатково ще такий оператор:

```
Vector2 operator+(const Vector2 &a, const Vector2& b); // [d]
```

не вдасться — він конфліктуватиме із `[a]`.

Перевантаження унарних операторів

- Аналогічно, унарний оператор може бути або нестатичним методом без аргументів, або незалежною функцією із одним аргументом.
- Тобто, використання довільного унарного оператора @: `@ x`, може інтерпретуватися як `x.operator@()`, або як `operator@(x)`:

```
class Vector2{
        ..........
    Vector2 operator−(); // [a] −−− унарний!
};
        ...........
Vector2 operator+(const Vector2& x); // [b] −−− унарний!
   ............
    Vector2 x, y;
        ..............
    x = −y; // Використовується [a], x = y.operator−();
    y = +x; // Використовується [b], y = operator+(x);
```

Перевантаження інкременту та декременту

- Цих операторів, "++" і "--", стосуються ті ж правила, що й інших унарних операторів.
- Постфіксні оператори перевантажуються за допомогою фіктивного аргументу типу int.

```
class A{
    ....
    A& operator++(); // Префіксний
    A operator++(int); // Постфіксний
    A& operator--(); // Префіксний
    A operator--(int); // Постфіксний
};
```

Також можна було реалізувати їх як незалежні функції.

- Зверніть увагу на тип, що повертається. Префіксний оператор повинен повертати lvalue, постфіксний — rvalue (тому що так є для фундаментальних типів!). Повертати посилання в першому випадку і об'єкт в другому – найпростіший спосіб цього досягнути[3]
- До C++29 додано =default для відповідних постфіксних операторів, який реалізовує їх типовим чином, через префіксні:

```
constexpr T operator++(int) = default;
```

[3] За умови потреби в хитрих оптимізаціях, можна змусити постфіксний оператор не повертати нічого. Скажімо так робила бібліотека математичних векторів blitz++. Однак воно збиває з пантелику, тому необхідність такого слід добре обдумувати.

Операції над дробами

- Розглянемо перевантаження операторів, необхідні, щоб наблизити зручність використання SimpleRatio до такої для вбудованих (фундаментальних) типів.
- Типи значення, особливо арифметичні, такі як дроби (а також — математичні вектори, комплексні числа, кватерніони, матриці, поліноми, тощо), мають певні напрацьовані схеми перевантаження операторів. За ними стоїть тривалий досвід, що дає хороші результати, що ні. Розглянемо типовий підхід до їх побудови[4]. Основна мотивація — природність та передбачуваність використання.
- Безвідносно до конкретних математичних класів, хорошою практикою є розумна мінімізація кількості методів і функцій-друзів, тобто, таких, що мають безпосередній доступ до внутрішнього представлення класу.
- Для лаконічності, в прикладах коду нижче, оголошуючи ті чи інші сутності, наводитиметься лише та частина класу, яка важлива для них, маючи на увазі, що все решта теж присутнє, і це не буде явно підкреслюватися за допомогою крапочок.

[4] Переваги саме таких рішень зазвичай обґрунтовуватимуться, однак не всюди це буде зробити просто, тому дещо доведеться брати на віру.

Контроль за інваріантами

- Дроби вважаються зведеними до канонічної — повністю скороченими, знаменник додатний, якщо чисельник 0 – знаменник рівний 1. Для контролю цього, особливо там, де воно критично – код покладається на інваріанти, використовуватимемо макроси assert. Для більшої зручності запропоную ряд допоміжним макросів:

```
#define ASSERT_CHECK_IF_REDUCED(obj) \
 assert (
   (obj).znam>0 && \
   (  (obj).chys == 0 || gcd(  std::abs((obj).chys),  (obj).znam )==1 )
)

#define ASSERT_CHECK_IF_REDUCED_THIS \
   ASSERT_CHECK_IF_REDUCED(*this);

#define ASSERT_CHECK_IF_REDUCED_BOTH do{ \
   ASSERT_CHECK_IF_REDUCED( *this );\
   ASSERT_CHECK_IF_REDUCED( rhs ); \
} while( false )
```

- В рамках зменшення кількості методів, які повинні мати доступ до внутрішнього представлення, виконуватимемо математичні операції в операторах "+=", "-=", "*=", "/=", а відповідні "+", "-", "*", "/" реалізуємо з їх допомогою.
- Крім того, швидше з естетичних міркувань, всі "+", "-", "*", "/", зробимо незалежними — природніше на додавання дивитися, як на `c = add(a, b);`, ніж як на `c = a.add(b);`.
- На прикладі додавання:

```
class SimpleRatio{
    SimpleRatio& operator += (const SimpleRatio& rhs);
    SimpleRatio& operator += (int rhs);
};
inline SimpleRatio operator+(SimpleRatio lhs, const SimpleRatio& rhs){
    return lhs += rhs;  }
inline SimpleRatio operator+(SimpleRatio lhs, int rhs){
    return lhs += rhs;  }
inline SimpleRatio operator+(int lhs, SimpleRatio rhs){
    return rhs += lhs;  }
```

Зауважте, що там де потрібна копія аргументу, замість створювати її явно ("`SimpleRatir res = lhs;`"), використовується передача за значенням.

Реалізація та використання додавання

- SimpleRatio& SimpleRatio::**operator** += (**const** SimpleRatio& rhs)
```
{
    chys = chys * rhs.znam + znam * rhs.chys;
    znam *= rhs.znam;
    reduce();
    return *this;
}
```
- Повертається посилання на той самий об'єкт, до якого було виконано додавання.
- Завдяки трьом перевантаженням `operator+` і двом `operator+=`, можливими є всі наступні комбінації:

```
SimpleRatio l, r, c;
int i;
    ..............
c = r + l;
c = r + i;
c = i + l;

c += r;
c += i;
```

Пройдемося ще раз по етапах розробки operator+:

- Прямолінійний, відверто неефективний, варіант:

```
inline SimpleRatio operator+(SimpleRatio lhs, SimpleRatio rhs){
    SimpleRatio res = lhs;
    res += rhs;
    return res;
}
```

- Передача за посиланням ефективніша, а так як += повертає посилання на свій лівий операнд, можна зекономити ще один рядок коду:

```
inline SimpleRatio operator+(const SimpleRatio& lhs,
                             const SimpleRatio& rhs){
    SimpleRatio res = lhs;
    return res += rhs;
}
```

- Однак, раз нам потрібна копія першого операнда, давайте довіримо її створення компілятору:

```
inline SimpleRatio operator+(SimpleRatio lhs, const SimpleRatio& rhs){
    return lhs += rhs;
}
```

- Звичайно, хороший компілятор і сам мав би позбутися таких, зайвих, копіювань, але покладатися на це не варто.
- З моєї практики, останній варіант не завжди швидший за перед-останній – залежить від компілятора, однак на загал, все ж краще зупинятися саме на ньому.

Винятки

- Реалізація @ через @= не є законом природи, хоч часто і зручна. Тому, якщо такий підхід заважає, від нього можна відмовлятися.
- Наприклад, множення матриць:

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} * \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}$$

  вимагає обох незмінних операндів (схожа ситуація буде, якщо під час додавання дробів, спершу змінити знаменник), тому простіше реалізовувати `operator*=` через `operator*`.
- З іншого боку, такий підхід настільки поширений, що в boost є бібліотека, Operators, котра, поміж іншого, по певному мінімальному комплекту, реалізовує для класу всі інші оператори. Детальніше див. `http://www.boost.org/doc/libs/1_57_0/libs/utility/operators.htm`.

- Унарні мінус та плюс реалізовано як методи — першому потрібен доступ до внутрішнього представлення[5], а другий просто для одноманітності:

```
class SimpleRatio{
    SimpleRatio operator-() const ;
     inline SimpleRatio operator+() const
    {
        return *this;
    }
};
    SimpleRatio SimpleRatio::operator-() const{
        SimpleRatio res(*this);
        res.chys = -res.chys;
        return res;
    }
```

- Зауважте, повертається саме копія, тому що у виразі:

```
b = -a;
```

аж ніяк не очікується, що значення a може бути змінено[6]!

[5] Можна обійтися за допомогою get_XXX, звичайно...

[6] Що не заважає писати подібні оператори авторам багатьох невдалих підручників C++.

Декілька допоміжних методів

```
class SimpleRatio{
    inline bool positive() const {
        assert (znam > 0);
        return chys >= 0;
    }
    inline bool negative() const{
        return ! positive ();
    }
    int sign() const;
    SimpleRatio& invert(); // Перетворює d в 1/d.
};
SimpleRatio& SimpleRatio::invert(){
    int temp = chys;      // Або використати функцію std::swap
    chys = znam;
    znam = temp;
    return *this;
}
int SimpleRatio::sign() const{
    assert (znam > 0);
#ifdef SIMPLE_RATIO_USE_FAST_SIGN
    return (chys > 0) - (chys < 0);
#else
    return chys>0 ? 1 : ( chys < 0 ? -1 : 0 );
#endif
}
```

Порівняння дробів — 1

- Операції рівності та співвідношення, реалізовано як методи — щоб спростити їм роботу, надавши безпосередній доступ до приватних полів:

```
class SimpleRatio{
    inline bool operator==(const SimpleRatio& rhs) const {
        ASSERT_CHECK_IF_REDUCED_BOTH
        return chys == rhs.chys && znam == rhs.znam;
    }
    inline bool operator==(int rhs) const {
        ASSERT_CHECK_IF_REDUCED_THIS
        return chys == rhs && znam == 1;
    }
    inline bool operator>(const SimpleRatio& rhs) const {
        ASSERT_CHECK_IF_REDUCED_BOTH
        return chys * rhs.znam > rhs.chys * znam;
    }
    inline bool operator>(int rhs) const {
        ASSERT_CHECK_IF_REDUCED_THIS
        return chys > rhs * znam;
    }
};
```

Порівняння дробів — 2

- Оператори, потрібні для симетрії — порівняння цілого числа та дробу, вимушено зроблені незалежними:

```
inline bool operator==(int lhs, const SimpleRatio& rhs) {
    return rhs == lhs;
}

inline bool operator>(int lhs, const SimpleRatio& rhs){
    // Протилежним до > є !(<=):
    // Якщо a > b то b <= a
    return rhs <= lhs;
}
```

З іншого боку, їх реалізація тривіальна.

Стандартні функції

- Крім різноманітних арифметичних операторів, SimpleRatio реалізовує ряд стандартних математичних функцій, присутніх у cmath, таких що раціональне число (дріб) залишають раціональним числом. Це функції від floor/ceil/round, та інших схожих, до pow(SimpleRatio, int). А ось, скажімо, тригонометричні функції чи піднесення до степенів типу double, не реалізовано — результат не буде раціональним числом.
- Реалізація їх є можливою в двох підходах:

  double sin(**const** SimpleRatio& arg);
  SimpleRatio sin(**const** SimpleRatio& arg);

  де другий варіант повертає дріб, найближчий до точного результату, однак, була залишена як вправа, для тих, у кого є відповідне натхнення.

Оператори вводу-виводу — 1

- Оператори вводу (>>) та виводу (<<) повинні бути симетричні — перший з них повинен вміти прочитати те, що вивів другий! (Зокрема, тому ніяких там "введіть чисельник" у них не повинно бути – для цього можна створити окремі функції.)
- Операція читання, яка враховує можливість читання у дріб звичайного цілого, може виглядати так:

```
std :: istream& operator>> (std::istream& os, SimpleRatio& z) {
  int ch, zn;
  if( os >> ch ) {
    if( os.peek() == '/') { // Підглядаємо, що там наступне в потоці
      char t;
      os >> t;
      if( os >> zn ) {
        z.chys = ch;     z.znam = zn;
      }
    } else {
      z.chys = ch;       z.znam = 1;
    }
  }
  return os;
```

Оператори вводу-виводу — 2

- Оператор виводу можна реалізувати наївно:

```
std::ostream& operator<< (std::ostream& os, const SimpleRatio& z) {
    os << z.chys;
    if (z.znam!=1)
        os << '/' << z.znam;
    return os;
}
```

- Однак, разом із маніпулятором setw(), який діє лише на наступний вивід[7] вона призведе до відверто поганого результату. Виправити це можна так:

```
std::ostream& operator<< (std::ostream& os, const SimpleRatio& z) {
    std::stringstream ss;
    ss << z.chys;
    if (z.znam!=1)
        ss << '/' << z.znam;
    os << ss.str();
    return os;
}
```

- Цей оператор все ще має багато недоліків, див. далі.

[7] Про потоки буде окрема велика розмова.

Підсумки та подальший розвиток

- Код повністю можна побачити у прикладі до цієї презентації, `l11_simple_rational`. Реалізація має багато цілком виправних недоліків. Зокрема:
- Реалізація арифметичних операцій прямолінійна, тому може допускати переповнення проміжних обчислень, навіть тоді, коли кінцевий результат є в межах представимих значень.
- Підтримка інваріантів вимагає великих обчислювальних ресурсів, через постійні виклики reduce(), до якої входить алгоритм Евкліда. Використання меташаблонного програмування (конкретніше, expression templates), разом із лінивими обчисленнями (laze evaluation), могло б значно збільшити продуктивність. Детальніше про підхід див., наприклад, бібліотеки blitz++[8] і eigen[9].
- Оператор виводу погано дружить із іншими видами форматування, типу виведення знаку "+" завжди, вирівнювання не пробілами, тощо. Акуратна реалізація виводу вимагатиме помітних зусиль. В меншій мірі це ж стосується і введення за допомогою `operator>>`.
- Бракує реалізації багатьох математичних функцій.
- Бракує повноцінних unit-тестів та (doxygen) документації.

[8] `http://blitz.sourceforge.net/` та списки розсилки там, так як сайт відверто застарілий.

[9] `http://eigen.tuxfamily.org/` та `http://indrekis2.blogspot.com/2012/07/eigen.html`

Об'єкти-функції або функтори

- Завдяки можливості перевантажувати оператор виклику функції, можна створювати класи, об'єкти яких поводяться як функції:

```
struct accum {
  accum(int isum) : sum(isum) {}
  accum& operator()(int x) { sum += x; return *this;}
  int get_sum() const { return sum; }
private:
  int sum;
};
  .....
    accum s(5);
    s(11);
    s(12);
    s(13)(14);
    cout << s.get_sum();
```

- Ця можливість широко використовується при роботі із STL, але має і самостійну цінність — на відміну від звичайних функцій, вони можуть містити стан, яким легко керувати за потреби.
- Єдиний з операторів, який дозволяє аргументи за замовчуванням.

Створення багатомірних масивів

Будувати багатомірні масиви можна двома способами:

- З використанням operator() замість operator[]: ar(1, 2, 3).
  - ◇ Цей спосіб найбільш зручний та гнучкий.
- Альтернативний варіант — створити operator[], який повертатиме допоміжний (proxy) клас, котрий міститиме масив, розмірності на одиницю меншої, ніж вихідний: ar[1][2][3].
  - ◇ Ймовірно, це найбільш проблемний спосіб – і для написання, і для оптимізатора[10].
- З використанням спеціальної функції ar.data(1, 2, 3).
  - ◇ Переваги над перевантаженням оператора () дуже сумнівні.

[10] Детальніше див. офіційний C++ FAQ `https://isocpp.org/wiki/faq/operator-overloading/3#matrix-array-of-array`, "Why shouldn't my Matrix class's interface look like an array-of-array?", та наступний пункт там же: "I still don't get it. Why shouldn't my Matrix class's interface look like an array-of-array?".

Створення багатомірних масивів – C++23

- В C++23 дозволено створення operator[] з багатьма аргументами[11]:

```
class X{
        .....................
  T& operator[](size_t i, size_t j, size_t k);
};
```

- Кількість аргументів може бути нульовою.
- ◇ В якомусь сенсі, ми отримали ще один синтаксис виклику функцій.

[11] Старе використання коми в ar[1,2,3] як оператора коми оголошено застарілим ще раніше, в C++20.

Перевантаження new та delete

Оператори new та delete можна перевантажувати, однак з їх перевантаженням є ряд нюансів.

- Оператор new дещо особливий.
- По перше, він може перевантажуватися із якою завгодно кількістю аргументів.
- По друге, можна перевантажити як глобальний new, так і new для конкретного класу (як його статичний метод).
- Зрозуміло, що перевантажуючи new, майже завжди слід перевантажити і відповідний delete.
- Хоча оператор new і оголошено в <new>, він є "вбудованим" — вважається оголошеним в кожній одиниці трансляції, безвідносно до того, включено <new>, чи ні. Крім того, вбудований new глобальний, не належить до простору імен std. Взагалі, поміщати перевантаження new в простори імен заборонено[12].

[12] Див. наприклад, відповідь тут: http://stackoverflow.com/questions/6210921/operator-new-inside-namespace.

- Компілятор оголошує наступні перевантаження new[13]:

```
void* operator new (std::size_t size);
void* operator new (std::size_t size, const std::nothrow_t& nt) noexcept;
void* operator new (std::size_t size, void* ptr) noexcept;

void* operator new[] (std::size_t size);
void* operator new[] (std::size_t size, const std::nothrow_t& nt) noexcept;
void* operator new[] (std::size_t size, void* ptr) noexcept;
```

- C++17 додав аналогічні перевантаження із вирівнюванням, (детальніше про них у відповідній лекції), виду:

```
void* operator new  ( std::size_t count, std::align_val_t al  );
void* operator new  ( std::size_t count, std::align_val_t al, const std::nothrow_t&  );
```

  Результат буде вирівняний на значення al. За замовчуванням, вирівнювання – `__STDCPP_DEFAULT_NEW_ALIGNMENT__`.

- Можна перевантажувати "звичайний" new та nothrow new, глобальний placement new перевантажувати не можна. Це ж стосується і їх варіантів для масивів, new[].
- Пам'ятаємо про різницю між виразом new та оператором new!
- std::nothrow — константа типу nothrow_t, використовується лише для вибору правильного оператора.

[13] До C++11, використовувалася застаріла throw() замість noexcept, а решта були помічені throw (std::bad_alloc).

- Можна створювати свої варіанти new:

  void* **operator new** (std::size_t size, **int** arg1, **const char*** arg2);

- Під час виклику new, першим аргументом йому передається необхідний розмір пам'яті в байтах (результат sizeof, фактично), наступними — решта, передані в дужках у відповідному виразі.

  int *p1 = **new**(nothrow) **int**;
  int *p2 = **new**(11, "abc") **int**;

- **Увага!** Перевантажувати глобальні new слід дуже обережно, зваживши всі плюси та мінуси! Таке перевантаження повністю заміщає глобальний вбудований оператор, і, як показує практика – майже ніколи не є повністю коректною, завжди щось ламає.
- Детальніше див. `http://en.cppreference.com/w/cpp/memory/new/operator_new` — формальніший, але більш строгий виклад, та більш доступний: `http://www.cplusplus.com/reference/new/operator%20new/`, `http://www.cplusplus.com/reference/new/operator%20new[]/`, та посилання там.

- Компілятор також оголошує "симетричні" варіанти delete:

```
void operator delete (void* ptr) noexcept;
void operator delete (void* ptr, const std::nothrow_t& nt) noexcept;
void operator delete (void* ptr, void* voidptr2) noexcept;

void operator delete[] (void* ptr) noexcept;
void operator delete[] (void* ptr, const std::nothrow_t& nt) noexcept;
void operator delete[] (void* ptr, void* voidptr2) noexcept;
```

- Однак, компілятор завжди викликає лише перший варіант delete/delete[]. Це ж стосується користувацьких delete, із додатковими аргументами, (які теж існують).
- Решта викликаються лише відповідним виразом new, якщо під час роботи конструктора сталося виключення.
- Звичайно, їх можна викликати явно за допомогою дещо дивного синтаксису:

```
operator delete[]( ptr, nothrow);
```

- Детальніше див. `http://en.cppreference.com/w/cpp/memory/new/operator_delete`, `http://www.cplusplus.com/reference/new/operator%20delete/` та посилання там.

Приклад (чисто ілюстративний) перевантаження ::new/::delete — 1

```
void* operator new(size_t num){
  void* res = malloc(num);
  if (!res){
    printf("Failed to allocate %zu bytes, using custom ::new()\n", num);
    throw std::bad_alloc();
  }
  printf("Allocated %zu bytes, at %p, using custom ::new()\n", num, res);

  return res;
}
void* operator new(size_t num, const std::nothrow_t & nt) throw(){
  void* res = malloc(num);
  printf("Allocated %zu bytes, at %p, using custom ::new(nothrow)\n", num,
  return res;
}
//! Custom new
void* operator new(size_t num, const char* comment) throw(){
  void* res = malloc(num);
  printf("Allocated %zu bytes, at %p, using custom ::new(%s)\n", num, res,
  return res;
}
```

Приклад (чисто ілюстративний) перевантаження ::new/::delete — 2

```
void operator delete(void *ptr) throw() {
    printf("Deleting at %p, using custom ::delete\n", ptr);
    free(ptr);
}
void operator delete(void *ptr, const std::nothrow_t& nt) throw(){
    printf("Deleting at %p, using custom ::delete(nothrow)\n", ptr);
    free(ptr);
}

void operator delete(void *ptr, const char* comment) throw(){
    printf("Deleting at %p, using custom ::delete(%s)\n", ptr, comment);
    free(ptr);
}
```

- Аналогічно перевантажені й new[]/delete[] — їх майже завжди теж потрібно перевантажувати, перевантаживши new/delete.
- Див. також приклад `l13_new_overloading_1` до цієї теми.

- Замість заміни глобального new, часто достатньо створити власний new для конкретного класу чи класів. Тоді він автоматично буде викликатися компілятором при виділенні динамічної пам'яті для об'єктів цього класу.
- Для цього слід оголосити new/delete як статичні методи класу.

```
class SampleND{
public:
  static void* operator new(size_t num);
  static void* operator new(size_t num, const std::nothrow_t& nt) noexcept;
  static void* operator new(size_t num, void *ptr) noexcept;

  static void operator delete(void *ptr) noexcept;
  static void operator delete(void *ptr, const std::nothrow_t& nt) noexcept;
  static void operator delete(void *ptr, void *ptr2) noexcept;

  static void* operator new[](size_t num);
  static void* operator new[](size_t num, const std::nothrow_t& nt) noexcept;
  static void* operator new[](size_t num, void *ptr) noexcept;

  static void operator delete[](void *ptr) noexcept;
  static void operator delete[](void *ptr, const std::nothrow_t& nt) noexcept;
  static void operator delete[](void *ptr, void *ptr2) noexcept;
    ..................
};
```

- Ці методи будуть статичні, навіть якщо не написати явно "static"!
- Тривіальний приклад реалізації new/delete для класу — див. приклад `l13_new_overloading_2` — в мінімальному варіанті вона не відрізняється від описаної вище, демонстративної, для глобальних операторів.
- Якщо всередині перевизначеного new є потреба виділити пам'ять за допомогою глобального оператора, можна скористатися наступним синтаксисом:

```
void* operator new(size_t sz){
  cerr << "new " << sz << " bytes\n";
  return ::operator new(sz);
}
```

- Якщо аналогічна потреба є в користувача класу [14], йому доведеться викручуватися нетривіально (див. приклад, `l13_new_overloading_2`).
- Часто варто перевантажувати всі варіанти оператора new, так як через особливості вибору функцій для перевантаження, зокрема – пошук лише в активній області видимості, один оголошений оператор new автоматично заборонить використовувати глобальні new інших видів. Детальніше див. також задачу 22 в "Новые сложные задачи на C++" Саттера.

[14] Що може свідчити про не до кінця продуманий дизайн класу.

- З міркувань можливості створення коду, безпечного щодо виключень, слід ніколи не випускати виключення за межі delete/delete[]!
- Деякі приклади використання перевантаження new можна знайти тут: `https://web.archive.org/web/20171204230256/https://www.relisoft.com/book/tech/9new.html`: кешування блоків пам'яті для повторного використання, виділення пам'яті цілими сторінками (chanks), пошук витікання пам'яті.
- Опис, як можна написати свій менеджер динамічної пам'яті: `https://developer.ibm.com/technologies/systems/tutorials/au-memorymanager`, єдине, стандартні менеджери пам'яті зараз помітно досконаліші, ніж натякає автор статті. Текст 2008 року – чи то я неправильно пам'ятаю, і тоді з ними було гірше, чи то автор тримав у голові ще старіші проблеми. Тому, стаття – хороша теоретична ілюстрація. Але щоб отримати практичну користь – потрібно заміряти продуктивність.

- Контейнери, зокрема, із стандартної бібліотеки, потребують пам'ять як для своїх потреб, так і для об'єктів, що зберігають. Для узгодженого керування її виділенням використовуються **алокатори**: `https://en.cppreference.com/w/cpp/memory/allocator`. C++17 додав ще поліморфні алокатори: `https://en.cppreference.com/w/cpp/memory/polymorphic_allocator`.
- Дві хороших презентації про алокатори: `http://ib-krajewski.blogspot.com/2021/12/my-two-talks-about-polymorphic-memory.html`.
- `https://en.cppreference.com/w/cpp/memory/shared_ptr/allocate_shared` – див. також алокатор для shared_ptr – алокує зразу і місце під службові дані (як і make_shared – останній (поки?) неможливо використовувати із кастомними алокаторами).

Конструктор за замовчуванням

Нагадаємо, що конструктор без аргументів використовується, коли створюється об'єкт відповідного класу без передачі жодних аргументів:

```
class A{
      .....
    A (){....}
};
 .....
     A a;
     A m[20];
```

Зокрема, він необхідний, якщо потрібно створювати масиви об'єктів — немає жодного способу передати їм аргументи.

Увага!, через особливості синтаксису C/C++ – все, що може вважатися функцією, вважатиметься функцією, наступний код:

```
A b();
```

буде не створенням об'єкту за допомогою конструктора за замовчуванням, а оголошенням функції з іменем b, типом значення, що повертається — A, та без аргументів.

Конструктори з одним аргументом

Конструктори, які можна викликати з одним аргументом[15] використовуватимуться для **неявного перетворення типів, із типу єдиного аргументу, в тип класу**. Це може приводити до неочікуваної поведінки:

```
class IntArray{
    size_t size;
     ......
    IntArray(size_t sz): size(sz) {....}
    int& operator[] (size_t index) {....}
};
 ......
     IntArray ar1(10); // OK, масив із 10 елементів
     IntArray ar2 = 10; // ? OK ? Теж створюємо масив із 10 елементів

     ar2 = 20; // ?!?! Створює новий масив із 20 елементів.

     ar1[i]==ar2; // Помилка! Однак, скомпілюється.
     // Компілятор використає неявний оператор перетворення,
     // створивши ще один масив розміром ar1[i], з яким і порівнюватиме!
```

explicit-конструктори

- Конструктори з одним аргументом використовуватимуться для неявного перетворення типів.
- Зокрема, вони використовуватимуться і при підборі функцій для перевантаження. Наприклад, у виразі "`ar2 = 20;`" не знаходиться оператора "=", який мав би першим аргументом `IntArray` а другим число, однак знаходиться "=", з двома аргументами "`IntArray`", який може бути вибрано із одним користувацьким перетворенням типів — все згідно правил.
- Це може приводити до небажаних наслідків.
- Щоб заборонити таке їх використання можна використати ключове слово explicit:

```
class IntArray{
      ......
   explicit IntArray(size_t sz): size(sz) {....}
};
```

- Воно забороняє використання такого конструктора в неявному перетворенні типів, хоча і залишає можливість використовувати його для явного перетворення, наприклад, за допомогою `static_cast`.
- Див. також "Most C++ constructors should be explicit" та приклад проблем тут: "Fun with flat_map's non-explicit constructors".
- ◇ Есей про проблеми відсутності explicit: "The case of creating new instances when you wanted to use the same one".

Умовний explicit C++20

- C++20 додав можливість умовного explicit(cond) – лише якщо умова часу компіляції, cond == true.
- Приклад:

```
struct S {
        ..............
  template <typename T>
   explicit (!std :: is_integral_v<T>) S(T)  {........}
```

- Див. приклади тут:
  `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0892r2.html`.

- Braced initialization із C++ 11 іноді дозволяє викликати приватний конструктор:

```
class Package {
  struct private_constructor { };

public:
  Package(int id, private_constructor); // Не викликати! Use CreatePackage
  static Package CreatePackage(int id, int flavor) { ..... } // Фабрика
};
  ...................
Package package(3, Package::private_constructor()); // Неможливо -- помилка.

Package package(3, {}); // А це -- скомпілюється...
```

- Допоможе тут explicite:

```
class Package {
  struct private_constructor
    { explicit private_constructor() = default; };
  ...........
```

- Приклад із `https://devblogs.microsoft.com/oldnewthing/20210719-00/?p=105454` – "C++11 braced initialization made the impossible possible (and how to fix it so it stays impossible)", розглянуто ситуацію, де конструктор має бути публічним, для сумісності із якимось зовнішнім кодом – фреймворком абощо, але не бажано, щоб його могли викликати безпосередньо.

Оператори неявного перетворення типів

- Конструктори із одним аргументом дозволяють здійснити перетворення з типу аргументу конструктора в тип класу.
- Для зворотного перетворення — з типу класу в якийсь інший, можна створювати спеціальні оператори:

```
class SimpleRatio{
        ..........
    operator double() const;
};
```

  які повертають значення того ж типу, що й ім'я оператора[16].
- Такі оператори викликатимуться автоматично для неявного перетворення типів.
- Краще їх уникати! Можливі сюрпризи, схожі на ті, що із конструкторами з одним аргументом.
- Альтернатива — явні методи для такого перетворення:

```
class SimpleRatio{
        ..........
    double to_double() const;
};
```

- Можуть бути віртуальними, але не статичними.
- Спосіб обмежити неявне перетворення: "Constrain your user-defined conversions".

[16] Без вказання типу значення, що повертається! Воно визначається назвою оператора.

Оператори неявного перетворення типів в C++14

- В C++14 можна також виводити тип оператора перетворення по типу виразу return. Однак, авто-виведення не відбуватиметься:

```
struct S {
  operator auto() const { return 10; } // OK
  template<class T> operator auto() const { return 42; } // error
  operator auto() -> short { return 42; } // error
};
```

explicit в C++11

- В C++11 можна оголосити explicit також оператори перетворення, заборонивши їх неявне використання:

```
class SimpleRatio{
        ...........
    explicit operator double() const;
};
```

- Важливо, що перетворення до bool, в контексті де явно потрібне булеве значення — if(), цикли, логічні операції, перетворення до bool вважається явним. Завдяки цьому, скажімо, можна зберегти звичне використання розумного вказівника в if(), не отримуючи разом із ним небажаних неявних перетворень в цілі числа:

```
class MySmartPointer{
        ...........
    explicit operator bool() const;
};
```

◇ Див. есей на тему: "What breaks without implicit T*-to-bool conversion?".

Delegating constructors (конструктори, які делегують) в C++11

- В C++98, якщо різні конструктори повинні робити одне і те ж, доводиться, щоб не дублювати код, створювати функцію типу init(), яку кожен із них викликає.
- В C++11 конструктор може використовувати інший конструктор:

```
SimpleRatio(int c, int z): chys(c), znam(z){
    reduce();
}
SimpleRatio(): SimpleRatio(0,1)
{}
explicit SimpleRatio(int c): SimpleRatio(c,1)
{}
```

- Об'єкт вважається сконструйованим, коли завершився "найбільш зовнішній" конструктор (а деструктор, скажімо, викликається лише для повністю сконструйованого об'єкта — якщо в будь-якому із ланцюжка конструкторів станеться виключення, деструктор не викличеться). Однак, при тому, на момент входу в тіло "найглибшого" конструктора, всі поля вже ініціалізовано, тому список ініціалізації конструкторів, що делегують частину роботи іншим конструкторам, не може містити нічого іншого, крім виклику конструктора. (Тіло, звичайно, може містити код, специфічний для цього конструктора.)

Ініціалізація в оголошенні класу — C++11

- Якщо деякі поля в більшості конструкторів повинні мати одне і те ж значення, зручніше вказати його безпосередньо під час оголошення. C++98 допускав таке лише для статичних констант інтегральних типів. В C++11 це можна робити практично для будь-яких типів:

```
struct S {
    int n = 7;
    double z = 7.3;
};
```

- Для ініціалізації класів із конструкторами, слід використати уніфіковану ініціалізацію (uniform initialization), те ж можна робити і для вбудованих типів, але для них це не обов'язково:

```
struct S {
    std :: string s{"abc"};
    int n1{77};
};
```

- Якщо котрийсь конструктор, в списку ініціалізації, містить поле, оголошене з ініціалізатором, цей ініціалізатор ігнорується.
- Для ініціалізації статичних полів не інтегральних типів, обов'язковим є використання constexpr:

```
struct S {
    constexpr static double d1=3.3;
};
```

Правило трьох/правило п'яти/правило нуля

- Якщо в класі є нетривіальний конструктор копіювання, оператор присвоєння, деструктор, (правило трьох – C++98), конструктор переміщення чи оператор присвоєння із переміщенням (правило п'яти, C++11) — майже завжди потрібні і всі решта!
- Однак, варто проектувати код так, щоб не потрібно було писати жоден з цих методів – правило нуля.
  - Наприклад, якщо всі члени даних класу мають відповідні методи – писати їх окремо в класі, що їх містить, потрібно рідко.

Посилання

- "Why does C++ think my class is copy-constructible when it can't be copy-constructed?" – есей про особливості визначення копіювабельності.
- "What happens when a destructor throws" – just don't.

POD — Plain Old Data

- Ідейно, це типи, сумісні за поведінкою і розташуванням в пам'яті із типами C.
- POD типи — це скалярні[17] типи, POD-класи та їх масиви. POD-класи потребують детальнішого розгляду.
- POD-класи це агреговані класи (POD-структури) та агреговані об'єднання (POD-об'єднання), які **НЕ мають**:
  - Нестатичних полів (включаючи масиви) типу вказівників на члени.
  - Нестатичних полів не-POD типів.
  - Нестатичних полів типу посилань.
  - Користувацьких операторів присвоєння чи деструкторів.

  При чому, агреговані класи, це класи, які **НЕ мають**:
  - Користувацьких конструкторів.
  - private чи protected нестатичних полів.
  - Базових класів.
  - Віртуальних функцій.

[17] Сюди входять фундаментальні, вбудовані типи, перерахування, вказівники, та вказівники на члени. Const/volatile можна додавати до смаку.

Властивості POD-типів

- POD типи мають наступні властивості:
  - Вони неперервні, і сумісні із відповідними C-ними структурами, що мають ті ж члени даних, в тому ж порядку.
  - Статичні змінні цих типів ініціалізуються нулями, автоматичні неініціалізовані явно – містять "сміття", невизначені значення.
  - Їх можна побайтово скопіювати, або в масив char/unsigned char, потім назад, або в інший такий же POD-об'єкт, без жодної зміни їх значення.
  - Вказівник на змінну POD-типу просто вказує на перший елемент (хоча розіменування і потребуватиме reinterpret_cast).

Тобто, зокрема, їх можна копіювати memcpy() чи memmove(), виділяти пам'ять malloc(), ініціалізувати memset().

POD в C++11

- В C++11 поняття POD розділили на дві незалежні частини: тривіальні типи, і типи із стандартним форматом (в пам'яті) — standard-layout.
- POD-типи, це типи, які одночасно є тривіальними і мають стандартний формат.
- Тривіальні класи повинні мати :
  - Тривіальний конструктор за замовчуванням.
  - Тривіальні конструктори копіювання та переміщення.
  - Тривіальні оператори присвоєння (як для копіювання, так і для переміщення).
  - Тривіальний, зокрема – **НЕ** віртуальний деструктор.
- Конструктор є тривіальним, якщо:
  - Створений компілятором – не програмістом.
  - Клас не містить віртуальних функцій.
  - Клас не має віртуальних базових класів.
  - Клас не має не-статичних членів з ініціалізаторами.
  - Конструктор базового класу – тривіальний.
  - Конструктори всіх не-статичних членів – тривіальні.
- Вимоги до тривіального деструктора аналогічні.

- Standard-layout класи:
  - НЕ мають віртуальних функцій чи віртуальних базових класів.
  - Всі нестатичні дані мають ті ж права доступу (public, private, protected).
  - Всі нестатичні поля і базові класи самі є standard-layout.
  - Всі нестатичні дані в ієрархії таких класів, оголошені в одному і лише одному із них:

```
struct B { int i; };        // standard-layout
struct C : B { };           // standard-layout
struct D : C { };           // standard-layout
struct E : D { char x; };   // НЕ standard-layout
```

  - Не має двох базових класів того ж типу:

```
struct A {};
struct X : A { };
struct Y : A { };
struct Z : X, Y { };     // НЕ standard-layout class
```

  - Не має базового класу того ж типу, що перше оголошене поле. Рекурсивно це ж стосується першого поля цього типу, його першого поля і т.д., якщо це не об'єднання. Для об'єднань це стосується всіх його полів[18].
- Тривіальні класи можна копіювати memcpy, однак вони можуть бути не сумісні із структурами C.
- Класи із стандартним форматом, у свою чергу, можуть мати складну структуру, конструктори, деструктори, і т.д., але при тому сумісні із аналогічними структурами C, з ними може працювати код C.

[18] Детальніше див. http://en.cppreference.com/w/cpp/concept/StandardLayoutType і посилання на Empty base optimization там.

- C++11 містить файл заголовків, <type_traits>, у якому є три шаблони, для діагностики:

  template <**class** T> **struct** is_pod;
  template< **class** T > **struct** is_trivial ;
  template< **class** T > **struct** is_standard_layout;

  які мають константне поле value, рівне true, якщо відповідне твердження справедливе для переданого типу, false, якщо ні:

```
struct A { int i; };          // Cструктура– (POD)
class B : public A {};        // ще POD немає( нових даних)
struct C : B { void fn(){} };// те ж
struct D : C { D(){} }; // Не POD –– є користувацький конструктор
    .....................
  cout << "int: " << is_pod<int>::value << endl;
  cout << "A: " << is_pod<A>::value << endl;
  cout << "B: " << is_pod<B>::value << endl;
  cout << "C: " << is_pod<C>::value << endl;
  cout << "D: " << is_pod<D>::value << endl;
```

  ◇ Але див. наступний слайд.

- Взагалі, <type_traits> містить багато цікавого, від is_arithmetic та is_integral, до is_copy_constructible та is_trivially_copy_constructible.

POD в C++20 і далі

- В C++20 поняття POD визнано застарілим[19] – замість нього залишилися поняття trivial і standard layout.
  - ◇ is_pod оголошено застарілим в C++20.
  - ◇ is_trivial в C++26 оголошено застарілим, на користь is_trivially_copyable, is_trivially_constructible, is_trivially_default_constructible, is_trivially_copy_constructible, is_trivially_move_constructible, is_trivially_assignable, is_trivially_copy_assignable, is_trivially_move_assignable, is_trivially_destructible. В автора цих презентацій цей факт викликає роздратування – навіщо більш дрібний поділ, зрозуміло, але навіщо викидати узагальнення?
- Дещо про нюанси: `https://quuxplusone.github.io/blog/2023/02/24/trivial-swap-x-prize/`. За певних умов, дані можуть зберігатися в хвостових padding байтах члена класу. Автор пропонує зміни до наступного стандарту, щоб дозволити оптимізації й в цьому випадку.
- "This-pointing Classes" – ілюстрація труднощів у випадку, коли клас містить явні чи неявні вказівники на свій ж вміст[20].

[19] Трохи подробиць: `https://mariusbancila.ro/blog/2020/08/10/no-more-plain-old-data/`.

[20] Власне, саме через такі випадки багато що в цій машинерії таке складне.

Trivial relocatability – C++26

- Trivial relocatability – більша підтримка для типів, які можна переміщати за допомогою memcpy:
  - ◇ Це такі об'єкти, які можна перемістити за допомогою memcpy (побайтово) і припинення часу життя яких не потребує виклику деструктора.
- Суміжна ідея – Replaceability, коли присвоєння з переміщення еквівалентне знищенню і move-створенню.
- Надаються способи вказати об'єкти як такі, що задовільняють відповідному типу, разом з можливістю компілятора виконувати відповідні оптимізації.
- Приклад trivially relocatable і replaceable: std::shared_ptr, std::future, std::vector. Також, std::pair і std::tuple будуть такими, якщо такими є їх типи елементів.
- std::tuple<T &> чи класи з const-членами є trivially relocatable, але не replaceable.
- Приклад replaceable, але не trivially relocatable – debug-ітератори, які відстежують свій контейнер, стрічки з SSO, які мають внутрішній вказівник.
  - ◇ Зауважте, вказівник на частину самого об'єкту ламає trivial relocatability.
- Синтаксис, разом з відповідними is_trivially_relocatable і is_trivially_relocatable_v тощо:

```
struct Y memberwise_trivially_relocatable {};
struct Z memberwise_replaceable {};
```

  - ◇ Також, додано функції trivially_relocate(), trivially_relocate_at(), std::relocate().
- Документ з пропозицією: "P2786R13: Trivial relocatability for C++26".

Неявно створювані методи класів

Навіть якщо написати наступний код:

```
class A
{ };
```

компілятор створить:

```
class A{
public:
    A();                                // Конструктор за замовчуванням
    ~A();                               // Деструктор за замовчуванням
    A(const A& rhs);                    // Конструктор копіювання
    A& operator=(const A& rhs); // Оператор присвоювання
    A* operator&();                     // Оператори отримання адреси
    const A* operator&() const;
};
```

Звичайно, реальний код для них генеруватиметься лише за потреби.
Однак, ідейно вони присутні завжди, включаючи POD-типи.

В C++11 створюються також:

```
A(A&& rhs);                 // Конструктор переміщення
A& operator=(A&& rhs); // Оператор присвоєння із переміщенням
```

- Наступні вирази для об'єкта b, фактично, тотожні, і викликають конструктор копіювання, а не присвоєння:

```
A a;
A b(a);
A b = a;
```

- Однак, для другого випадку оператор присвоєння таки повинен існувати, просто компілятор має право опускати його виклик для оптимізації. (Що може стати проблемою, якщо конструктор чи оператор присвоєння мають побічні ефекти). Див. також `http://en.wikipedia.org/wiki/Copy_elision` та `http://en.wikipedia.org/wiki/Return_value_optimization`.
- Починаючи із C++17, використання copy elision стало значно більш обов'язковим, див. `http://en.cppreference.com/w/cpp/language/copy_elision`.
- Згідно моїх експериментів, станом на весну 2018, всі основні компілятори для x86, під час компіляції без оптимізації, виконують copy elision навіть без C++17.
- Однак, щодо філософськи-прагматичного сенсу такої зміни див. `https://akrzemi1.wordpress.com/2018/05/16/rvalues-redefined/`.
- Потенційний конфлікт цієї оптимізації з escape analysis в сучасних (2021) реалізаціях компіляторів: `https://quuxplusone.github.io/blog/2021/03/07/copy-elision-borks-escape-analysis/`.
- Приклад copy elision з `https://devblogs.microsoft.com/oldnewthing/20210628-00/?p=105374`: `Buffer b = Buffer(Buffer(Buffer(1)));` еквівалентно `Buffer b(1);`.

- Конструктор за замовчуванням викликає конструктори за замовчуванням для всіх типів, однак НЕ ініціалізує поля фундаментальних типів[21].
- Конструктор за замовчуванням не створюватиметься, якщо є константні члени даних чи посилання — їх слід ініціалізувати явно.
- Деструктор за замовчуванням не робить нічого. Він, одначе, буде віртуальним, якщо базовий клас даного має віртуальний деструктор.
- Обидва оператори взяття адреси просто повертають адресу об'єкта: "`return this;`".
- Конструктор копіювання та оператор присвоєння почленно копіюють нестатичні члени даних. Якщо така процедура неможлива — відповідні неявні методи не створюються[22].
- Якщо явно створено хоч один конструктор відповідного типу, компілятор перестає автоматично генерувати інші. Наприклад, якщо є заданий конструктор із обов'язковими аргументами, конструктор за замовчуванням більше не створюється.

[21] За деякими нюансами див. http://stackoverflow.com/questions/563221/is-there-an-implicit-default-constructor-in-c

[22] Наприклад, оператор присвоєння не зможе модифікувати константи та посилання в тому об'єкті, в який здійснюється присвоєння.

- Для того, щоб позбутися якихось із автоматично створених методів, можна використати =delete.
- Наприклад, щоб заборонити копіювання[23]

```
class A{
    A(const A& rhs) = delete;
    A& operator=(const A& rhs) = delete;
      ..............
};
```

- Його можна застосовувати до всіх методів, що існують за замовчуванням.
- Також, так можна контролювати дозволені перевантаження, детальніше про таке застосування – у розмові про шаблони[24].
- До C++11 схожого результату досягали, оголосивши відповідні методи в приватній частині класу і не надавши реалізації.
- C++11 визнав застарілим автоматичне створення `https://quuxplusone.github.io/blog/2023/05/05/deprecated-copy-with-dtor/`, `https://eel.is/c++draft/depr.impldec`, компілятори можуть давати попередження про це.

[23] Це може бути потрібно, якщо клас має складну ініціалізацію, наприклад виділяє пам'ять, відкриває файли, тощо, і копіювання є накладним, або й взагалі позбавленим сенсу.

[24] Приклад використання для рефакторингу: `https://quuxplusone.github.io/blog/2022/11/11/refactoring-with-delete/`.

=default — C++11

- Навпаки, якщо вам потрібен метод за замовчуванням, а компілятор перестав його генерувати, замість переписувати самому, можна змусити компілятор передумати:

```
class A{
    A() = default;
    A(int x);
      ..............
};
```

- Такий метод вважається явно створеним – іноді це впливає на результат. Див., наприклад, "Once more about the rule of 5" і "The rule of 5 and inheritance".

Додатково про =delete — C++11+

- =delete можна застосовувати до будь-яких методів:

```
struct NoInt {
    void f(double i);
    void f(int) = delete;
};
```

тепер, замість неявного перетворення до int, компілятор просто відмовиться викликати f() із цілим аргументом.

- Так само можна робити і з шаблонами методів:

```
struct OnlyDouble {
    void f(double d);
    template<class T> void f(T) = delete;
};
```

- Див. також `https://quuxplusone.github.io/blog/2021/10/17/equals-delete-means/` – "What '=delete' means".
- C++26 обіцяє додати `=delete("Reason for deletion")` – можливість додати текстовий опис причини видалення, який повідомлятиме компілятор за спроби використати видалену конструкцію.

Узгодженість копіювання

- Якщо є потреба створити один із наступних методів: конструктор копіювання, конструктор присвоєння, деструктор, то дуже ймовірно, що потрібно створити і два інших. (Чотири інших в C++11 — відповідні конструктор та оператор присвоєння для переміщення).
- Варто використовувати стандартну форму операторів копіювання та присвоєння:

```
class A
{
    .....
    A(const A& rhs);
    A& operator=(const A& rhs);
};
```

```
class ArrayT{
    double *data;
    size_t size;
public:
    ArrayT(size_t sz):
      size(sz){
        data = new double[size];
    }
    ~ArrayT(){
        delete[] data;
    }

    ArrayT& operator=(const ArrayT& rhs);

    ArrayT(const ArrayT& rhs):
        size(rhs.size){
        data = new double[rhs.size];
        for(size_t i=0;i<rhs.size;++i){
            data[i] = rhs.data[i];
        }
    }
};
```

Варіант операції присвоєння

```
ArrayT& ArrayT::operator=(const ArrayT& rhs){
    // if( this != &rhs ){ <== Не потрібне!
        double *new_data = new double[rhs.size];
        for(size_t i=0;i<rhs.size;++i){
            new_data[i] = rhs.data[i];
        }
        delete[] data;

        data = new_data;
        size = rhs.size;
    // }
    return *this;
}
```

- Зауважте, що перевірки на присвоєння самому собі немає. Справа в тому, що в нормі таке присвоєння і не має відбуватися — зазвичай воно свідчить про недогляд. Звичайно, повинен працювати коректно навіть в такому випадку! Але турбуватися про швидкодію сенсу немає.
- Зокрема, код вище коректно, хоч і повільно, відпрацьовує присвоєння самому собі. Так як таке присвоєння — аномалія, а за перевірку "`if( this != &rhs )`" платитимуть всі користувачі. Тому вона зайва!

swap

- Код вище має суттєвий недолік — дублювання коду[25].
- Щоб його позбутися, слід створювати тимчасовий об'єкт, ініціалізуючи його копіюванням, а потім обміняти з поточним об'єктом (який буде очищено його деструктором):

```
void swap(ArrayT& lhs, ArrayT& rhs){
    size_t ts = rhs.a_size;
    rhs.a_size = lhs.a_size;
    lhs.a_size = ts;

    int* td = rhs.data;
    rhs.data = lhs.data;
    lhs.data = td;
}

ArrayT& ArrayT::operator=(const ArrayT& rhs){
    ArrayT temp(rhs);
    swap(temp, *this);
    return *this;
}
```

[25] Ще один можливий недолік був би, якби типом елементів був не double, а клас, що може кидати виключення під час копіювання. У випадку такого виключення, відбувалося б витікання пам'яті. Як з тим боротися — пізніше.

- Хороша стаття – коли які методи створювати: `https://foonathan.net/2019/02/special-member-functions/`.
- Картинка-шпаргалка з неї (сама по собі – не дуже корисна), взята звідси: "How I Declare My class And Why)"[26]:

Special Members

compiler implicitly declares

user declares	default constructor	destructor	copy constructor	copy assignment	move constructor	move assignment
Nothing	defaulted	defaulted	defaulted	defaulted	defaulted	defaulted
Any constructor	not declared	defaulted	defaulted	defaulted	defaulted	defaulted
default constructor	user declared	defaulted	defaulted	defaulted	defaulted	defaulted
destructor	defaulted	user declared	defaulted	defaulted	not declared	not declared
copy constructor	not declared	defaulted	user declared	defaulted	not declared	not declared
copy assignment	defaulted	defaulted	defaulted	user declared	not declared	not declared
move constructor	not declared	defaulted	deleted	deleted	user declared	not declared
move assignment	defaulted	defaulted	deleted	deleted	not declared	user declared

[26] Див. також презентацію автора, звідки вона взята.

- Табличка із: `https://www.cppstories.com/2022/non-regular-members/`, про те, які методи створюються, якщо клас має певні нерегулярні члени даних:

Non-static data member type	Default ctor	Copy ctor	Copy assign	Move ctor	Move assign
copyable, assignable, “regular”	yes	default	default	default	default
`const` data member	no, unless default value is set	default	custom only	default	custom only
pointer type	yes	default(shallow!)	default(shallow!)	default	default
`std::unique_ptr`	yes	custom only	custom only	default	default
`std::shared_ptr`	yes	default, shallow, but might be safe	default, also shallow	default	default
reference type	no	default(shallow)	custom only	default	custom only
`std::reference_wrapper`	no	default (shallow!)	default(shallow)	default	default

Посилання

- `https://devblogs.microsoft.com/oldnewthing/20210916-00/?p=105693` – "The C++ implicit assignment operator is a non-ref-qualified member, even if the base class's assignment has a ref-qualifier"
- `https://quuxplusone.github.io/blog/2021/09/17/a-class-without-a-copy-constructor/` – як позбутися конструктора копіювання...
- "Excessive copying in C++ and your program's speed"
- "Growing Buffers to Avoid Copying Data"

- Існує таке поняття, як вказівник на член класу (як член даних, так і функцію-член). Воно, зокрема, використовується для роботи із одноманітними членами, вибір яких слід робити динамічно, під час виконання коду — як альтернатива switch.
- Вказівники на члени не є вказівниками у звичному розумінні! Це своєрідні індекси всередині класу[27]. Для їх використання необхідний об'єкт! Детальніше див. відповідний розділ офіційного C++ FAQ: `https://isocpp.org/wiki/faq/pointers-to-members`
—

[27] Для полів це, зазвичай, зміщення відносно початку класу, для методів — просто вказівник на відповідну функцію.

Вказівники на методи

- Синтаксис вказівника на метод (функцію-член) виглядає так:

  ReturnType (ClassName::* pointer_name) (...ArgumentList ...);

- Можливо, на прикладах буде більш зрозуміло:

```
struct IntObj{
    int val;
    IntObj(int x=0):val(x){}
    IntObj& add(IntObj rhs){ val += rhs.val; return *this;}
    IntObj& div(IntObj rhs){ val /= rhs.val; return *this;};
    IntObj& sub(IntObj rhs){ val -= rhs.val; return *this;};
    IntObj& mul(IntObj rhs){ val *= rhs.val; return *this;}; };
.................
IntObj a(1), b(2);      cout << a.add(b).val << endl;

IntObj& (IntObj::*memberPtr)(IntObj);

memberPtr = &IntObj::add;
(a.*memberPtr)(b);              cout << a.val << endl;

memberPtr = &IntObj::sub;
(a.*memberPtr)(b);              cout << a.val << endl;

IntObj *pa = &a;
(pa->*memberPtr)(b);            cout << a.val << endl;
```

Вказівники на поля

- Синтаксис вказівників на поля (члени даних) дещо простіший:

  FieldType ClassName::* pointer_name;

- Приклад:

```
struct AssortiCollector {
    AssortiCollector (): dogs(0), cats(0), foxes(0){}
    int dogs;
    int cats;
    int foxes;
};
.....
    AssortiCollector collection ;

    int AssortiCollector ::*dataPtr;

    dataPtr = &AssortiCollector :: cats;
    collection .*dataPtr += 11;

    dataPtr = &AssortiCollector :: dogs;
    collection .*dataPtr += 2;

    AssortiCollector * collectionPtr = &collection;
```

- Синтаксис вказівників на методи (і, в меншій мірі, на поля даних) доволі складний у використанні.
- Тому краще користуватися відповідними using чи typedef (pre-C++11):

```
using memberPtrType = IntObj& (IntObj::*)(IntObj);
    // або:
typedef IntObj& (IntObj::*memberPtrType)(IntObj);

memberPtrType a1;
a1 = &IntObj::add;
(a.*memberPtr2)(b);

using dataPtrType = int  AssortiCollector  ::*;
    //або
typedef int  AssortiCollector  ::*dataPtrType;
dataPtrType d1;
d1 = &AssortiCollector :: dogs;
 collection .*dataPtr2 += 2;
```

- Дужки навколо `(a.*memberPtr2)` у виклику `(a.*memberPtr2)(b)` необхідні! Аналогічно, необхідне взяття адреси, "&", в присвоєнні.
- Вказівник на константний метод:

```
using const_mem_ptr = res_t (IntObj::*)(Arg1T, Arg2T) const;
```

- Масив вказівників на методи:

```
IntObj& (IntObj::*memberPtrArray[10])(IntObj);
memberPtrArray[0] = &IntObj::add;
(a.*memberPtrArray[0])(b);
```

- Вкладеність вказівників на члени класів не обмежена — можуть існувати вказівники на члени, які у свою чергу, є вказівниками на члени, і т.д:

```
struct PtP {
    int m;
    int PtP::* const p; // const pointer to non-const member
};
.....
    const PtP cPtPobj = {1, &PtP::m};
    // non-const pointer to member which is a const pointer to non-con
    int PtP::* const PtP::* ptcp = &PtP::p;
    std::cout << cPtPobj.*(cPtPobj.*ptcp) << endl;

    // regular non-const pointer to a const pointer-to-member
    int PtP::* const * ptcp2 = &cPtPobj.p;
    std::cout << cPtPobj.**ptcp2 << '\n'; // prints 1
```

Приклад взято тут:
http://en.cppreference.com/w/cpp/language/pointer

- Вказівники на константні методи:

```
class IntObj{
    ..............
     int unity() const {return 1;};
};
    ..............
typedef int (IntObj::*memberPtrNoArgsType)() const;
```

- Статичні методи, фактично, є незалежними функціями, доступними в контексті класу, тому вказівник на них виглядатиме так:

```
struct IntObj{
    ..............
     static int sf(){ return 1; };
     static double dx;
};

double IntObj::dx=3.17;
    ..............
     int (*cstatiMethodPtr)() = &IntObj::sf;
     cstatiMethodPtr();

     double *staticDataPtr = &IntObj::dx;
```

Поліморфізм

- Вказівники на функції працюють поліморфно для віртуальних функцій:

```
struct Base{
    virtual void fn() { cout << "I'm from Base" << endl; }
};

struct Derived: public Base{
    virtual void fn() { cout << "I'm from Derived" << endl; }
};
    .................
    Base bs1;
    Derived dr1;

    typedef void (Base::*memberPtrToBaseType)();
    memberPtrToBaseType basePtr = &Base::fn;

    (bs1.*basePtr)();
    (dr1.*basePtr)();
```

Перетворення вниз та вверх по ієрархії

- Правило для присвоєння вказівників на методи в ієрархії класів протилежне до звичного — можна присвоювати вказівнику на метод нащадка вказівник на метод предка, але не навпаки[28]. Це ж стосується вказівників на поля даних.

```
typedef void (Base::*memberPtrToBaseType)();
typedef void (Derived::*memberPtrToDerivedType)();
memberPtrToBaseType basePtr = &Base::fn;
memberPtrToDerivedType derivedPtr = &Derived::fn;

derivedPtr = basePtr;          //! OK
//! basePtr = derivedPtr;      //! Error!
```

- За потреби таке перетворення можна зробити за допомогою `static_cast<basePtr>`:

```
basePtr = static_cast< void (Base::*)() >(derivedPtr); //! Forced OK
basePtr = static_cast<memberPtrToBaseType>(derivedPtr); //! Forced C
```

Таке перетворення компілюється, навіть якщо в предка відповідної функції немає... Однак, зрозуміло, це груба помилка.

[28] Предкова функція буде в нащадку завжди, а от нащадкова в предку — ні.

static і constexpr

- Розглянемо наступний фрагмент коду:

```
constexpr auto get_val_free() { return 0; }
struct test_static_constexpr_init{
    static constexpr int x1{0}; // OK
    static constexpr auto get_val() { return 0; }
    // static constexpr int x2{ get_val() }; //! Error!
    struct impl_t{ static constexpr auto get_val() { return 0; } };
    // static constexpr int x3{ impl_t::get_val() }; //! Error!
    static constexpr int x4{ get_val_free() }; // OK
    static constexpr int x5{ []() { return 0; }() }; // OK
    static constexpr int x6{ []() constexpr { return 0; }() }; // OK
    // static constexpr int x7{ []() constexpr { return get_val(); }()
};
```

- Як видно, статичну constexpr функцію не можна використовувати для ініціалізації навіть статичних констант часу компіляції.
- Clang повідомляє про це відносно зрозуміло:

```
error: constexpr variable 'x3' must be initialized by a constant expression
note: undefined function 'get_val' cannot be used in a constant expression
```

Справа в тому, що статична функція вважається визначеною коли завершено визначення класу.

offsetof

- Макрос offsetof із <cstddef> дозволяє отримувати зміщення поля структури чи класу відносно його початку.
- Для standard-layout типів перший член завжди має зміщення нуль.
- Для не POD/не standard-layout типів до C++17 використовувати – невизначена поведінка, в C++17 – підтримка на розсуд реалізації, потрібно повідомляти про помилку, якщо не реалізовано.

```
struct S {
    char   a;
    double b;
    short  c;
    char   d;
    float  e[10];
};

cout << offsetof(S, c) << '\n'
cout << offsetof(S, e) << '\n'
cout << offsetof(S, e[5]) << '\n'
```

Посилання

- Есей про час життя об'єктів, за умови NRVO та того, що частина з них – константні: `https://akrzemi1.wordpress.com/2023/12/18/the-double-life-of-objets/`.
- Structs and constructors – дещо дискусійний есей на тему конструкторів у (простих) структур.





